

Monitoreo de CPU y Kernel en PostgreSQL 17

Guía de Instalación, Configuración de postgresql.conf y Consultas SQL

Esta guía contiene el procedimiento paso a paso para habilitar el seguimiento de consumo de CPU de nivel de kernel (OS) y eventos de espera mediante la extensión `pg_stat_kcache` y `pg_stat_statements` en PostgreSQL 17.

PASO 1 Paquetes OS a Descargar e Instalar

Debes descargar e instalar el paquete binario oficial de la extensión correspondiente a tu distribución y versión de PostgreSQL:

En Debian / Ubuntu:

```
sudo apt-get update
sudo apt-get install postgresql-17-pg-stat-kcache
```

En RHEL / CentOS / Rocky Linux / AlmaLinux:

```
sudo dnf install pg_stat_kcache_17
# O usando yum:
sudo yum install pg_stat_kcache_17
```

PASO 2 Parámetros a Configurar en postgresql.conf

Edita el archivo de configuración principal de PostgreSQL (`postgresql.conf`) para cargar las librerías necesarias en memoria compartida:

Parámetro	Valor Recomendado	Descripción / Requerimiento
<code>shared_preload_libraries</code>	<code>'pg_stat_statements, pg_stat_kcache'</code>	Requiere reinicio. El orden importa: <code>pg_stat_statements</code> va primero.
<code>compute_query_id</code>	<code>on</code>	Genera los IDs unívocos (<code>query_id</code>) para cruzar vistas.
<code>track_activities</code>	<code>on</code>	Permite rastrear sesiones y eventos en <code>pg_stat_activity</code> .
<code>track_activity_query_size</code>	<code>2048</code> (o mayor)	Aumenta la longitud capturada de las consultas SQL.

¡IMPORTANTE! Reinicio del Servicio:

Después de modificar `shared_preload_libraries`, debes reiniciar PostgreSQL para surtir efecto:

```
sudo systemctl restart postgresql
```

PASO 3 Habilitar Extensiones en la Base de Datos

Conéctate a tu base de datos mediante `psql` o tu cliente SQL y ejecuta:

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pg_stat_kcache;
```

PASO 4 Scripts SQL de Monitoreo y Pruebas

1. Consulta de Monitoreo en Tiempo Real (JOIN pg_stat_activity + pg_stat_kcache)

Obtiene las sesiones activas, interpretando `NULL` como ejecución activa de CPU y cruzando el uso de CPU/I/O real registrado por el OS:

```
SELECT
    a.pid,
    a.username AS usuario,
    a.datname AS base_datos,
    a.state AS estado,
    COALESCE(a.wait_event_type, 'CPU') AS wait_event_type,
    COALESCE(a.wait_event, 'CPU Executing') AS wait_event,
    ROUND(CAST(k.exec_user_time AS numeric), 2) AS cpu_user_seconds,
    ROUND(CAST(k.exec_system_time AS numeric), 2) AS cpu_system_seconds,
    ROUND(CAST((k.exec_user_time + k.exec_system_time) AS numeric), 2) AS cpu_total_seconds,
    pg_size_pretty(k.exec_reads) AS disco_lectura_real,
    pg_size_pretty(k.exec_writes) AS disco_escritura_real,
    a.query_id,
    a.query AS consulta_actual
FROM pg_stat_activity a
LEFT JOIN pg_stat_kcache_detail k
    ON a.query_id = k.queryid
   AND a.usysid = k.userid
   AND a.datid = k.dbid
WHERE a.state = 'active'
      AND a.pid != pg_backend_pid();
```

2. Función PL/pgSQL para Simular Consumo de CPU (\$100\%\$ de un Núcleo)

Utiliza cálculos matemáticos iterativos para probar el monitoreo durante un tiempo determinado:

```
CREATE OR REPLACE FUNCTION simular_carga_cpu(segundos integer DEFAULT 10)
RETURNS text AS $$
DECLARE
    tiempo_limite timestamp;
    resultado double precision := 0.0;
    iteraciones bigint := 0;
BEGIN
    tiempo_limite := clock_timestamp() + (segundos || ' seconds')::interval;
    WHILE clock_timestamp() < tiempo_limite LOOP
        resultado := sin(iteraciones::double precision) * cos(iteraciones::double precision)
            + sqrt(iteraciones::double precision + 1.0);
        iteraciones := iteraciones + 1;
    END LOOP;
    RETURN format('Simulación finalizada. Duración: %s seg. Iteraciones: %s', segundos, iteraciones);
END;
$$ LANGUAGE plpgsql;

-- Ejemplo de ejecución (15 segundos de carga intensiva de CPU):
SELECT simular_carga_cpu(15);
```